

REVISÃO PROVA: Algoritmos e Programação Estruturada

Sequência

Decisão

Repetição

O que é um algoritmo?

Uma sequência finita de passos bem definidos para resolver um problema

Ele descreve como sair de uma entrada e chegar a uma saída.

Ele não depende obrigatoriamente de uma linguagem de programação.

Pode ser escrito em texto, fluxograma, pseudocódigo ou código real.

Exemplos rápidos

- fazer café
-
- destravar o celular
-
- calcular média
- logar em um sistema

Precisão

Cada passo precisa ser claro e sem ambiguidade.

Finitude

O processo deve terminar após um número finito de etapas.

Efetividade

As operações descritas precisam ser executáveis na prática.

Entrada e saída

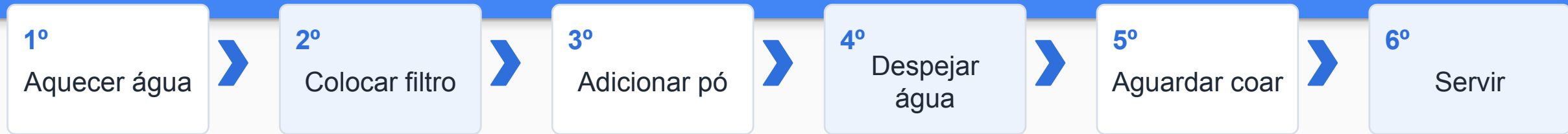
Recebe dados e produz um resultado observável.

Generalidade

Serve para uma classe de problemas, não apenas um caso isolado.

Exemplo do cotidiano: fazer café coado

do mundo real para a lógica



- Perceba a ordem: trocar etapas altera ou destrói o resultado.
- Se faltar um passo, o algoritmo fica incompleto.
- Programar é formalizar esse raciocínio em uma linguagem precisa.

Programação estruturada

Resolver problemas por composição de blocos simples e previsíveis.

1. Sequência

Instruções executadas na ordem em que aparecem.

2. Seleção

Escolha entre caminhos com base em uma condição.

3. Repetição

Execução reiterada enquanto uma condição pede continuidade.

Variáveis começam com \$.

A linguagem é dinamicamente tipada, mas tipos continuam importando no raciocínio.

```
1 <?php
2 $nome = "Ana";
3 $mensagem = "Olá, $nome";
4
5 echo $mensagem;
```

aritméticos: + - * / % **

comparação: == === != < > <= >=

lógicos: && || !

```
1 <?php
2 $preco = 250.00;
3 $desconto = 10;
4 $valorDesconto = $preco * ($desconto / 100);
5 $total = $preco - $valorDesconto;
6
7 echo $total;
```

A variável guarda estado durante a execução.

Operadores conectam os dados e definem o comportamento do algoritmo.

1. Confundir = (atribuição) com == / === (comparação).

2. Misturar números e strings sem perceber as conversões.

3. Esquecer ponto e vírgula ao final da linha.

4. Dar nomes pobres para variáveis e perder clareza.

```
1 <?php
2 $media = 7;
3
4 if ($media = 7) {
5     echo "Aprovado?";
6 }
```

Pergunta

- O que esse if realmente faz?

Leitura de código: rastreando valores

passo a passo

```
1 <?php
2 $preco = 250.00;
3 $desconto = 10;
4 $valorDesconto = $preco * ($desconto / 100);
5 $total = $preco - $valorDesconto;
```

- 1. O algoritmo lê cada linha.
- 2. Atualiza o estado das variáveis.
- 3. Só depois interpreta o resultado.

BLOCO 2

Decisão e repetição

- Código deixa de ser uma lista estática.
- O algoritmo começa a escolher e repetir.

Estruturas condicionais: if / elseif / else

```
classificação por média

1 <?php
2 $media = 6.8;
3
4 if ($media >= 7) {
5     echo "Aprovado";
6 } elseif ($media >= 5) {
7     echo "Recuperação";
8 } else {
9     echo "Reprovado";
10 }
```

A primeira condição verdadeira interrompe a cascata.

A ordem das condições importa. Condições precisam ser mutuamente claras e completas.

Quando a ordem das condições importa

evitando pegadinhas lógicas

Versão problemática

```
ordem ruim

1 if ($media >= 5) {
2     echo "Recuperação";
3 } elseif ($media >= 7) {
4     echo "Aprovado";
5 }
```



Versão correta

```
ordem certa

1 if ($media >= 6) {
2     echo "Aprovado";
3 } elseif ($media <= 5) {
4     echo "Recuperação";
5 }
```

switch case

```
1 <?php
2 $opcao = 2;
3
4 switch ($opcao) {
5     case 1:
6         echo "Cadastrar";
7         break;
8     case 2:
9         echo "Listar";
10        break;
11    default:
12        echo "Opção inválida";
13 }
```

while

repete enquanto a condição for verdadeira

do while

executa pelo menos uma vez antes de testar

for

ideal quando sabemos a contagem

foreach

perfeito para percorrer listas e arrays

Regra de ouro

- Laço sem condição de parada vira laço infinito.

while

```
1 <?php
2 $i = 1;
3
4 while ($i <= 5) {
5     echo $i . " ";
6     $i++;
7 }
```

Bom para cenários em que a condição controla o fluxo.

Quase sempre precisa de uma variável de controle.

ao menos uma
execução

```
<?php
$opcao = "sair";

do {
    echo "Bem-vindo ao sistema!";
} while ($opcao !== "sair");
```

Comparação mental

- while testa antes. do while testa depois.

for

```
1 <?php
2 for ($i = 0; $i < 5; $i++) {contagem controlada
3     echo $i . " ";
4 }
```

Três partes do for

inicialização

condição

incremento ou decremento

Muito usado para percorrer índices numéricos.

Excelente quando a quantidade de repetições é conhecida.

Evita espalhar controle de contador pelo bloco.

foreach

array indexado

```
1 <?php
2 $notas = [7, 8, 10];
3
4 foreach ($notas as $nota) {
5     echo $nota . " ";
6 }
```

array associativo

```
1 $produtos = [
2     "SKU-100" => "Notebook Dell",
3     "SKU-201" => "Mouse sem fio",
4 ];
5
6 foreach ($produtos as $codigo => $nome) {
7     echo "$codigo | $nome";
8 }
```

O foreach utilizado para percorrer arrays com menos atrito e mais legibilidade.

BLOCO 3

Arrays: ordenar, filtrar e transformar

- Sair do “um valor por vez”.
- Pensar em coleções de dados.
- Usar funções prontas do PHP com propósito.

Arrays em PHP: indexados e associativos

indexado

```
1 $notas = [7, 8, 10];  
2 // índices: 0, 1, 2
```

associativo

```
1 $aluno = [  
2     "nome" => "Carlos",  
3     "nota" => 8,  
4     "idade" => 20  
5 ];
```

Quando usar cada um?

indexado: listas e sequências

associativo: registros com campos

nomeados

crescente

```
1 $numeros = [5, 2, 9, 1];  
2 sort($numeros);  
3 print_r($numeros);
```

decrescente

```
1 $numeros = [5, 2, 9, 1];  
2 rsort($numeros);  
3 print_r($numeros);
```

preservando o original

```
1 $original = [5, 2, 9, 1];  
2 $copia = $original;  
3 sort($copia);
```

- `sort()` reorganiza o próprio array.
- `rsort()` faz a ordenação decrescente.
- Se você não quiser perder a ordem original, trabalhe em uma cópia.

Ordenação em arrays associativos: asort e ksort

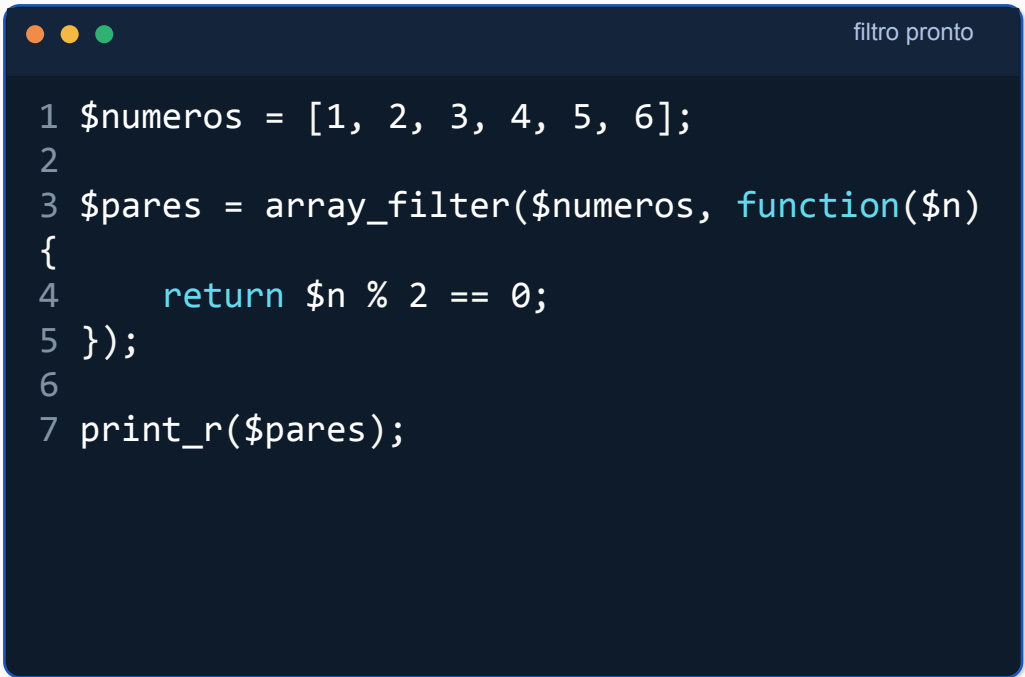
```
asort
1 $aluno = [
2     "nome" => "Carlos",
3     "nota" => 8,
4     "idade" => 20
5 ];
6
7 asort($aluno); // ordena pelos valores
```

```
ksort
1 $aluno = [
2     "nome" => "Carlos",
3     "nota" => 8,
4     "idade" => 20
5 ];
6
7 ksort($aluno); // ordena pelas chaves
```

- Quando o que importa é a etiqueta do dado, eu ordeno pela chave.

```
1 $numeros = [1,2,3,4,5,6];
2 $pares = [];
3
4 foreach ($numeros as $n) {
5     if ($n % 2 == 0) {
6         $pares[] = $n;
7     }
8 }
9
10 print_r($pares);
```

- O laço percorre o array inteiro.
- A condição decide quem entra.
- Um novo array guarda apenas os elementos aprovados.



```
1 $numeros = [1, 2, 3, 4, 5, 6];
2
3 $pares = array_filter($numeros, function($n)
4 {
5     return $n % 2 == 0;
6 });
7 print_r($pares);
```

Resumo do mecanismo

- PHP passa cada elemento para a função.
- Se retornar TRUE, o item fica.
- Se retornar FALSE, o item sai.

Vantagem didática

- menos código repetido, mais foco na regra de seleção

transformação

```
1 $numeros = [1, 2, 3, 4, 5];  
2  
3 $dobro = array_map(function($n) {  
4     return $n * 2;  
5 }, $numeros);  
6  
7 print_r($dobro);
```

- array_filter escolhe quem fica.
- array_map muda a forma de todos os elementos.
- Mapeamento é muito comum em listas de preços, nomes, notas e relatórios.

BLOCO 4

Funções e decomposição de problemas

- Sair do código monolítico.
- Criar blocos reutilizáveis e testáveis.
- Raciocinar por camadas.

- Reduzem repetição de código.
- Facilitam testes e manutenção.
- Melhoram legibilidade.
- Permitem raciocinar por camadas.
- Ajudam a localizar erros e efeitos colaterais.

Conceitos-chave

- parâmetro: valor recebido
- retorno: resultado produzido
- escopo: variável local não vaza sozinha

```
1 function saudacao($nome) {  
2     return "Olá, $nome";  
3 }
```

mini exemplo

Parâmetros, retorno e escopo

Parâmetro

- É a entrada da função.

Retorno

- É o resultado devolvido pela função.

```
função simples
1 function somar($a, $b) {
2     $resultado = $a + $b;
3     return $resultado;
4 }
5
6 echo somar(2, 3);
```

Passar parâmetro não é o mesmo que imprimir diretamente.

Retornar valor não é o mesmo que usar echo.

```
1 function calcularMedia(array $notas)
2 {
3     $soma = 0;
4
5     foreach ($notas as $nota) {
6         $soma += $nota;
7     }
8
9     return $soma / count($notas);
10 }
11
12 echo calcularMedia([8, 10, 9]);
```

- Recebe um array como parâmetro.
- Percorre os elementos com foreach.
- Acumula soma e retorna a média.

Ponto didático

- A função esconde detalhes e deixa o código principal mais limpo.

```
1 function compararNumeros($num1, $num2)
2 {
3     if ($num1 > $num2) {
4         return "O primeiro número é maior.";
5     } elseif ($num2 > $num1) {
6         return "O segundo número é maior.";
7     }
8
9     return "Os números são iguais.";
10 }
```

- A função recebe dois parâmetros.
- Cada ramo resolve um cenário lógico.
- O retorno evita depender de echo dentro da função.

Observação docente

- Em sala, vale comparar esta versão com a do slide-base para discutir melhoria de estilo.

```
idade + autorização

1 function verificarAcesso($idade, $autorizado)
2 {
3     if ($idade >= 18 && $autorizado) {
4         return "Acesso autorizado";
5     } elseif ($idade < 18) {
6         return "Acesso negado. Idade mínima: 18
anos";
7     }
8
9     return "Acesso negado. Autorização
necessária";
10 }
```

- Se a pessoa tiver 17 anos e autorização, qual ramo executa?

```
com match
1 function calcularDesconto($valor, $categoria)
2 {
3     $desconto = match ($categoria) {
4         'eletronicos' => 0.10,
5         'vestuario'   => 0.20,
6         'alimentos'   => 0.05,
7         default       => 0
8     };
9
10    return $valor - ($valor * $desconto);
11 }
```

- O retorno já entrega o valor final calculado.